STAFF WALKTHROUGH · RUNBOOK

Stock Market Analysis & Price Prediction

A practical script for presenting this project live — what order to show things in, exactly what to say, and how to recover if something breaks. Total time: ~12–15 minutes.

The 90-Second Pitch (say this first, before touching the keyboard)

> "This project takes real stock market data — either live from Yahoo Finance or an offline snapshot of nearly 6,000 tickers — and runs it through a full data science pipeline: cleaning, feature engineering with real technical indicators like RSI and MACD, training five different machine learning models and automatically picking the best one, and forecasting future prices. Everything's exposed through an 8-page interactive dashboard. I'll show it live."

How This Was Built

Built in Python 3.12 as a modular pipeline (`src/`) following the **CRISP-DM** methodology — the standard process model for data-mining projects: Business Understanding → Data Understanding → Data Preparation → Modeling → Evaluation → Deployment. Each stage is its own file with a single responsibility, orchestrated end-to-end by `main.py` (command line) and `app.py` (Streamlit dashboard) — both call the exact same underlying functions, so nothing behaves differently between the two.

LAYER	TOOL	ROLE
Data	pandas, numpy	Cleaning, feature math, resampling
Data source	yfinance	Live Yahoo Finance OHLCV, any ticker
Offline source	Kaggle dataset (pulled via `kagglehub`)	Static snapshot for reproducibility & scale
Visualization	Plotly, Matplotlib	15 interactive/static charts per ticker
ML	scikit-learn	5 regressors, GridSearchCV, TimeSeriesSplit
Persistence	joblib	Best model saved per ticker

Interface	Streamlit	8-page dashboard, same logic as the CLI

How It Works — the pipeline, stage by stage

- **Collect** — pull daily Open/High/Low/Close/Adj Close/Volume for any ticker, live or offline (identical schema either way).
- **Clean** — forward/back-fill missing values, drop duplicate dates, enforce numeric/date types, sort chronologically, flag (not drop) statistical outliers with the IQR method.
- **Engineer features** — derive 20 columns beyond raw price: daily return, 20/50/100-day moving averages, rolling volatility, RSI, MACD (+ signal + histogram), Bollinger Bands, and 3 lagged closes.
- **Explore** — generate 15 charts per ticker (trend, volume, candlestick, indicator overlays, distributions, correlation, autocorrelation) and summary statistics.
- **Model** — train 5 regressors (Linear Regression, Decision Tree, Random Forest, Gradient Boosting, linear-kernel SVR) on an 80/20 *chronological* split — never shuffled, since a time-series model must be tested on days it's never seen. The three tree-based models are hyperparameter-tuned with GridSearchCV over a TimeSeriesSplit cross-validator, so even the inner validation folds respect time order.
- **Evaluate** — MAE / MSE / RMSE / R^2 for every model; the best by R^2 is auto-selected and saved. Three diagnostic plots (actual-vs-predicted, residuals, prediction error over time) catch problems R^2 alone would hide.
- **Predict** — single-day prediction feeds the model its latest feature row. Multi-day forecasting iteratively simulates forward: each predicted close becomes the next day's assumed input, and every indicator is recomputed at each step — explicitly labeled as an approximation, not a guarantee.
- **Deploy** — the Streamlit dashboard exposes every stage above interactively, with caching so switching pages doesn't re-download or re-train.

About the Dataset — two sources, same schema

Both produce identical columns (`Date, Open, High, Low, Close, Adj Close, Volume`), so every downstream step works unchanged regardless of source.

	LIVE — YFINANCE	OFFLINE — KAGGLE BULK SNAPSHOT
Coverage	Any ticker Yahoo Finance recognizes (AAPL, TSLA, RELIANCE.NS, …)	~5,865 stocks + ~2,147 ETFs (`jacksoncrow/stock-market-dataset`)
Recency	Up to today	Frozen at 2020-04-01 — some tickers go back to the 1960s

Needs internet?	Yes	No, once downloaded
Best for	Current analysis, live demos	Reproducible grading, cross-ticker Market Overview page

A third mode, `--source auto` (or the dashboard's "auto" option), tries live data first and silently falls back to the offline copy if the network call fails — good insurance for a live demo.

Two real data-quality issues found and fixed in the offline dataset: some legacy tickers had corrupted prices from decades of stock-split back-adjustment (filtered with a minimum valid-price threshold), and raw total return wasn't comparable across tickers with wildly different history lengths, fixed by ranking on **CAGR** (annualized return) instead — the statistically correct normalization.

Before You Start — checklist

- Terminal open, virtual environment activated (`.venv\Scripts\activate`)
- Internet connection confirmed (for the live yfinance demo)
- Pick a ticker nobody's seen you run before — proves it isn't cached/rigged. Have a backup ticker ready too.
- This runbook and the **Class Notes PDF** open on a second screen/tab for reference
- `streamlit run app.py` pre-warmed in a spare terminal tab (starts faster when you actually need it)

Suggested Order

1 ~2 min

Run the full pipeline live, from a blank slate

```
python main.py --ticker <fresh-ticker> --years 5 --forecast-days 5
```

SAY: *Watch what happens — download, clean, engineer 20 features, train and tune 5 models, evaluate, forecast. All of it, one command, under a minute.*

While it runs, narrate each bracketed step [1/7]…[7/7] as it prints — this is the moment that proves the pipeline is real, not a canned demo.

2 ~1 min

Open the dashboard

```
streamlit run app.py
```

SAY: *Everything I just ran on the command line, staff can also drive interactively — pick any ticker, any date range, live or offline data.*

3 ~5 min

Walk the 8 pages, one line each

PAGE	ONE LINE TO SAY
Dashboard	"Snapshot: latest price, 52-week range, volume, and the trend — this is the landing view."
Stock Analysis	"Same data, filterable by date range, fully interactive — zoom, pan, hover for exact values."
Technical Indicators	"Candlestick chart plus the indicators real technical analysts use: Bollinger Bands, RSI, MACD."
EDA	"Statistics, correlation, and — this is the fun one — return autocorrelation, which tests market efficiency directly."
Prediction	"Click Predict — it forecasts forward using whichever model won for this ticker, with a clear caveat that it's a simulation, not a guarantee."
Model Performance	"All 5 models, ranked by R^2, with the tuned hyperparameters shown — nothing hidden."
Compare Tickers	"Overlay 2+ stocks normalized to the same starting point — see who actually outperformed."
Market Overview	"This one only works because of the offline dataset — cross-sectional stats across ~6,000 tickers at once, something no live API demo could do in real time."

4 ~2 min

Tell the two "bug" stories — this is what shows judgment, not just execution

SAY: *An early version scored an SVR model at R^2 of −2.6 — worse than useless. The cause: an RBF kernel can't predict a price higher than any it's ever seen. Switching to a linear kernel fixed*

it — R^2 0.9967, best model on Apple.

SAY: *Separately, in the offline dataset, some 1960s-era stocks showed "returns" in the millions of percent. Not a bug in the data — raw total return isn't comparable across 5-year vs 58-year histories. Fixed by ranking on CAGR instead.*

These two stories are worth more than any slide — they show you found real problems and diagnosed the actual cause, not just patched symptoms.

5 ~1 min

Close with proof it's reproducible

SAY: *It's all on GitHub — anyone can clone it, run one command, and get the same result.*

```
git clone https://github.com/kaushikbuilds-cloud/stockmarket.git
```

Anticipated Questions

Q: Is this actually predicting the future, or just fitting to history?

A: Single-day prediction is close to "tomorrow looks like today plus a learned adjustment." Multi-day forecasting is an explicit, stated simulation — say that before they ask.

Q: Why does a simple Linear Regression sometimes beat Random Forest?

A: Trees can't predict a price outside the range they trained on. On a trending stock, that's fatal — linear models extrapolate, trees plateau.

Q: Why two data sources?

A: Live for current data, offline for reproducibility with zero internet and for the cross-sectional Market Overview page a single live ticker can't support.

Q: What would you add with more time?

A: LSTM/GRU sequence models, news sentiment, portfolio risk metrics (Sharpe ratio, drawdown), scheduled retraining.

Q: How long did this take to build?

A: Answer honestly based on your own timeline — staff will respect an honest answer more than a vague one.

> **IF THE LIVE DEMO BREAKS**
>
> Wifi dies → switch the sidebar's **Data Source** to "offline" and keep going, uninterrupted — this is a good moment to say "this is exactly why the offline mode exists."
>
> Streamlit won't start → fall back to the **PPTX deck** or the **Class Notes PDF**, both already have real screenshots/numbers from actual runs — nothing about the fallback looks improvised.
>
> Someone asks something you don't know → "Good question, let me check the code and follow up" beats guessing, every time.